

Generating Natural Language From Logic Expressions With Structural Representation

Xin Wu^{ID}, Yi Cai^{ID}, *Member, IEEE*, Zetao Lian, Ho-fung Leung^{ID}, *Senior Member, IEEE*, and Tao Wang^{ID}

Abstract—Incorporating logic reasoning with deep neural networks (DNNs) is an important challenge in machine learning. In this article, we study the problem of converting logical expressions into natural language. In particular, given a sequential logic expression, the goal is to generate its corresponding natural sentence. Since the information in a logic expression often has a hierarchical structure, a sequence-to-sequence baseline struggles to capture the full dependencies between words, and hence it often generates incorrect sentences. To alleviate this problem, we propose a model to convert Structural Logic Expressions into Natural Language (SLEtoNL). SLEtoNL converts sequential logic expressions into structural representation and leverages structural encoders to capture the dependencies between nodes. The quantitative and qualitative analyses demonstrate that our proposed method outperforms the seq2seq model, which is based on the sequential representation, and outperforms strong pretrained language models (e.g., T5, BART, GPT3) with a large margin (28.6 in BLEU3) in out-of-distribution evaluation.

Index Terms—Natural language generation, logic expressions, Tree-LSTM, graph convolutional networks.

I. INTRODUCTION

LOGIC expression is a type of formal language which plays an important role in many computer science applications,

Manuscript received 19 November 2021; revised 24 May 2022, 2 October 2022, and 17 March 2023; accepted 22 March 2023. Date of publication 4 April 2023; date of current version 20 April 2023. The work of Tao Wang was supported by the Maudsley Charity and Early Career Research Award from the Institute of Psychiatry, Psychology and Neuroscience, King's College London. This work was supported in part by the National Natural Science Foundation of China under Grant 62076100, in part by the Fundamental Research Funds for the Central Universities, SCUT, under Grant x2rjD2220050, in part by the Science and Technology Planning Project of Guangdong Province under Grant 2020B0101100002, and in part by CAAI-Huawei MindSpore Open Fund. The associate editor coordinating the review of this manuscript and approving it for publication was Dr. Preslav Nakov. (*Corresponding author: Tao Wang.*)

Xin Wu and Zetao Lian are with the School of Software Engineering, South China University of Technology, Guangzhou 510006, China, and also with the Key Laboratory of Big Data and Intelligent Robot, Ministry of Education, South China University of Technology, Guangzhou 510006, China (e-mail: sexinwu@mail.scut.edu.cn; zettaolian@gmail.com).

Yi Cai is with the School of Software Engineering, South China University of Technology, Guangzhou 510006, China, and also with the Key Laboratory of Big Data and Intelligent Robot, Ministry of Education, South China University of Technology, Guangzhou 510006, China, and also with the Guangdong Artificial Intelligence and Digital Economy Laboratory, Pazhou Lab, Guangzhou 510335, China (e-mail: ycai@scut.edu.cn).

Ho-fung Leung is with the Department of Computer Science and Engineering, The Chinese University of Hong Kong, 999077, Hong Kong (e-mail: lhf@cuhk.edu.hk).

Tao Wang is with the Department of Biostatistics and Health Informatics, Institute of Psychiatry, Psychology and Neuroscience, King's College London, WC2R 2LS London, U.K. (e-mail: tao.wang@kcl.ac.uk).

The data and code will be available on <https://github.com>.

Digital Object Identifier 10.1109/TASLP.2023.3263784

Logic expression (input):

exists x01 . (driver (x01) & motorcycle (x01) & exists e02 . (dress (e02) & (acc (e02) = x01) & exists x03 . (orange (x03) & in (e02 , x03))))

Natural sentence (output):

a motorcycle driver dressed in orange. ✓

Seq2seq output:

a motorcycle driver in orange dress. ✗

Graph structure:

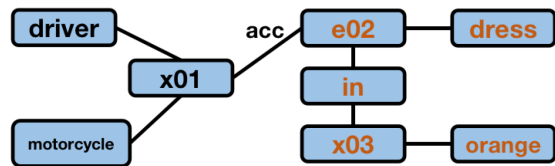


Fig. 1. The logic expression and graph structure of the sentence “a motorcycle driver dressed in orange”. “acc(e03) = x01” means “x01” is the accusative of “e03”.

including automated reasoning [1] and logic programming [2], software and hardware formal verification [3], [4]. Moreover, with the recent success of deep learning in many Natural Language Processing (NLP) tasks, logical expressions have also been integrated into Deep Neural Networks (DNN). For instance, DNNs are employed to construct logical expressions from images and texts, and then use a theorem prover to draw logical conclusions from the logic expressions [5], [6], [7]. Although logic expressions are useful for machines, they are difficult for humans without specialized knowledge to read, which hinders human understanding of the machine’s reasoning process.

To narrow the gap between formal logic expressions and natural language, we take a first step towards converting logical expressions into natural language in this article. Straightforward methods include training an end-to-end model through the seq2seq [8], [9] algorithm or fine-tuning pretrained language models [10], [11], [12]. However, according to our observations, there are two limitations as follows.

Firstly, the seq2seq model struggles to capture the dependencies between words, and hence it often generates incorrect sentences. As shown in Fig. 1, the seq2seq model is expected to

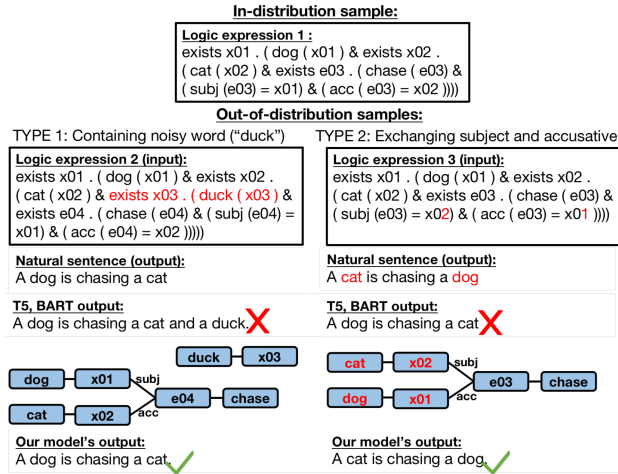


Fig. 2. Two types of out-of-distribution samples. The differences between in-distribution and out-of-distribution samples are marked in red. The pretrained language models fail to capture these changes and generate incorrect output.

generate sentence “a motorcycle driver dressed in orange” from the logic expression. Since “dress,” “orange,” and their relation “in” are far away in the logic expression, the seq2seq model does not capture their dependency, which results in generating an incorrect sentence “a motorcycle driver in orange dress”.

According to our observations, the information in logic expression is well-structured, and humans can construct a structural representation (e.g., tree and graph) to help understand the dependencies between words. As shown in Fig. 1, “dress” and “orange” has a directly connection to “in” through their corresponding variables “e02” and “x03”. From the graph, we can understand that their relationships are “dress in orange” rather than “in orange dress”. However, it is difficult for a seq2seq model to automatically induce the original dependencies from the sequential forms. To capture the dependencies between words in a logic expressions, it is necessary to represent the logic expressions as a well-defined structure (i.e., tree or graph).

Secondly, sequential models including state-of-the-art pretrained language models (e.g., T5, BART, GPT3) are not robust to out-of-distribution logic expressions. As illustrated by two examples in Fig. 2, there are two types of out-of-distribution samples. In Logic expression 2, “duck” is a *noisy* word since it has no connection to other words, and hence it should be ignored in the natural sentence. In Logic expression 3, the subject and accusative *exchanged* with a minor change. Ideally, a model should be robust to these changes when transforming logic expression to natural language. However, according to our observation, models based on sequential input fail to capture these changes and thus generate incorrect sentences. As illustrated in Fig. 2, T5/BART generates a sentence which contains the noisy word “duck” (logic expression 2) and a sentence with incorrect subject and accusative (logic expressions 3). In contrast, these changes can be easily captured in the structured representation and can help models to identify the corresponding features and generate correct sentences. Moreover, it would be easy to remove the noisy words in structure representation if they have no connection to others.

In this article, we propose a neural-network-based model to convert Structural Logic Expressions into Natural Language (SLEtoNL). Specifically, our model contains two phases: conversion phase and generation phase. In the conversion phase, a logic expression is converted to two types of structural representations (i.e., tree or graph). For the tree structure, we convert a sequential logic expression into a tree structure by a recursive parsing algorithm. For the graph structure, we merge the duplicate nodes in a tree and then convert it into a graph structure. In the generation phase, we use two types of structured encoders to full capture the structural representations and the dependencies between nodes. The first one is a Tree Long Short-Term Memory Networks (Tree-LSTM), which consists of a bottom up Tree-LSTM and a top down Tree-LSTM; The other structural encoder is Graph Convolutional Network which is to encode the graph structure input. Finally, a sequential decoder with the copy mechanism is applied to generate sentences. We construct a parallel dataset to evaluate our proposed method. The quantitative (including automatic and human evaluation) and qualitative analyses demonstrate that our method outperforms the seq2seq, which is based on the sequential representation, and outperforms the state-of-the-art pretrained language models with a large margin (28.6 in BLEU3) in an out-of-distribution evaluation.

The contributions of this article are three-fold:

- To narrow the gap between formal logic expressions and natural language, we propose a neural network based model to automatically convert logic expressions into natural sentences. Our model represents logic expressions with tree or graph structures, which can alleviate the difficulty in capturing the dependencies between words in a sequential logic expression.
- We construct a dataset for training and evaluation of the transformation from logic expressions to natural language. The dataset contains 11,000 sentences and their corresponding logic expressions. We conduct comprehensive experiments to evaluate our proposed model. The quantitative and qualitative analyses demonstrate the effectiveness of our proposed model.
- We conduct experiments to demonstrate the effectiveness of structured representations in handling out-of-domain samples, where our model outperforms the state-of-the-art pretrained language models (e.g., GPT3) with a large margin (28.6 in BLEU3).

The rest of this article is organized as follow: In Section II, we introduce the related work on logical text generation and AMR-to-text generation. In Section III, we introduce the problem definition and data construction procedures. In Section IV, we present our model in details. Section V describes details about the experiments. The conclusion is drawn in Section VI.

II. RELATED WORK

Natural Language Generation: Natural Language Generation (NLG) is a long-standing problem [13], [14]. In the most widely-cited survey of NLG methods to date [15], NLG is defined as the computer systems that can produce understandable texts

in English or other human languages from some underlying non-linguistic representation of information, including dialogue system [16], [17], story generation [18], [19], etc. Recently, several NLG studies investigate varieties of challenges, including personality [20], sentiment [21], and fidelity [22], [23].

AMR2text: One line of research related to ours is the text generation from Abstract Meaning Representation (AMR) [24], called AMR-to-text generation [25], [26]. While initial work used statistical approaches [27], [28], recent AMR-to-text research has demonstrated the success of deep learning, including using seq2seq [29], tree2seq [30], and graph2seq [31], and graph transformer [32]. Both logic expressions and AMR are formal language, and can represent the meaning of language in a precise and formal way. The main difference between AMR and logic expressions lies in their structures and syntaxes, which lead to different expressive capabilities [33], [34], [35]. Specifically, AMR does not contain information on tense, aspect, number, and articles, and cannot express universal quantification [24]. In contrast, logic expressions has a more powerful expressive capability than AMR, but it is also more complex. Due to the limitations of AMR in its expressive abilities, it is usually only used as an auxiliary representation embedded within NLP models [36], [37] and cannot directly perform symbolic reasoning. On the other hand, logic expression is a symbolic language that allows for direct symbolic reasoning and has been widely used in previous studies [38], [39], making it more suitable for our goal of converting machine symbolic reasoning results into natural language. Therefore, we ultimately chose logic expressions.

Structure2text: [40] propose a tree-to-sequence model and utilize it coupled with a better constrained decoding method to achieve better semantic correctness in Task-Oriented Dialog. [41] propose the Graph-to-Sequence model which captures the global structure information of the SQL query well in SQL-to-text task. [32] present the Graph Transformer based entirely on automatic attention which enables global node-to-node communication in global graph structure modeling.

Logical NLG: Recently, several NLG studies investigate generating logical natural language from the structured data, called logic2text [22] and logical natural language generation [23]. There are two differences between their studies and ours. The first one is that they are designed for tabular data and cannot be applied to logical reasoning. The second difference is that their logic focuses on count, superlative, comparative, and majority, while our goal is to investigate the performance of NLG with more general logic (not only numerically logic).

III. DATA CONSTRUCTION AND PROBLEM DEFINITION

In this section, we firstly formally define our proposed generation task. Then we introduce how to construct the parallel dataset for evaluation, including a normal in-distribution set and a challenging out-of-distribution set.

A. Problem Definition

Given an input logic expression $L = x_1, x_2, \dots, x_l$, where l is the length of the logic expression. our task is to train a model to

generate a natural language sentence $Y = y_1, y_2, \dots, y_n$, where n is the length of the sentence.

Each logic expression is described in the Higher-Order Logic (HOL) formula, which is a formal logic statement with quantifiers, predicates, and functions that operate on abstract objects. Each logic formula may contain:

- Variables - placeholders for concrete objects. e.g., x, y
- Predicates - functions that evaluate to true or false for given arguments. e.g., $\text{red}(x)$, $\text{black}(y)$
- Connectives - logical operators between predicates (and, or, not). e.g., $(\text{red}(x) \ \& \ \text{black}(y))$
- Quantifiers - that introduce variables into a formula. e.g., Forall x . Exist y .

For example, the formula “for all x , $P(x)$ ” states that the property P holds for all objects x .”

B. Data Construction

1) *In-Distribution Set*: We firstly obtain natural language sentences and then annotate their logic expressions. We collect 12,000 sentences from SNLI¹ dataset [42]. However, annotating logic expression for each sentence is time-consuming and requires expert knowledge. One possible approach is to use a semantic parser to obtain logic expressions from natural language sentences. ccg2lambda [5] is a simple and easy-to-use semantic parser to produce logical semantic representations of sentences. The parsing processing has two stages.. The first stage is to parse input sentences into Combinatory Categorical Grammar (CCG) trees. ccg2lambda currently supports the C&C parser [43] for English. The second stage is semantic composition, where ccg2lambda uses lambda calculus to compositionally constructed output logic expressions (possibly higher-order). One example is shown in Fig. 2, where the logic expression contains two entities ($x01$ and $x03$) and one event ($e02$). The “ $\text{acc}(e03) = x01$ ” means “ $x01$ ” is the accusative of “ $e03$ ”. The “ $\text{in}(e02, x03)$ ” means “ $e02$ ” and “ $x03$ ” have a connection named “in”.

However, the logic expression constructed by ccg2lambda contains several meaningless characters, including “.” and “_”. We apply several regular expressions to clean the logic expressions before the next procedure. We collect 11,000 sentences and their logic expressions, which contains 9,000 for training, 1,000 for development, and 1,000 as test set. The statistics of the dataset are illustrated in Table II. The logic operators including “exists” (existential quantification), “all” (universal quantification), “&” (conjunction), “|” (disjunction), “-” (implication) and so on. Table I shows two examples of the dataset. In the first example, the logic expression contains two entities ($x01$ and $x02$) and one event ($e01$). The “ $\text{subj}(e03) = x01$ ” means “ $x01$ ” is the subject of “ $e03$ ”. The “ $\text{acc}(e03) = x02$ ” means “ $x02$ ” is the accusative of “ $e03$ ”. Similarly, in the second example, the logic expression contains two entities and two events. “ acci ” here means the accusative of “ $e02$ ” is not a single entity, but a phrase (see [5] for more details on the logic expressions). To examine data quality, we manually verify 100 random samples, and only

¹[Online]. Available: <https://nlp.stanford.edu/projects/snli/>

TABLE I
TWO EXAMPLES OF OUR DATASET

No	Logic Expression (input)	Natural Language (output)
1	exists x01.(_man(x01) & exists x02.(_bread(x02) & exists e03.(_make(e03) & (subj(e03) = x01) & (acc(e03) = x02))))	a man is making bread
2	exists x01.(_man(x01) & exists e02.(_go(e02) & (subj(e02) = x01) & acc(e02, exists x03.(_building(x03) & exists e04.(_enter(e04) & (subj(e04)=x01) & (acc(e04)=x03))))))	the man is going to enter a building

TABLE II
SUMMARY OF THE CONSTRUCTED DATASET

Split	Count	Avg. Sent. Leng.	Avg. Expr. Leng.	Avg. Logic. Num.
Train	9000	44.01	76.43	13.22
Dev	1000	43.88	76.19	13.18
Test	1000	43.60	76.60	13.31

“Avg. Sent. Len.” indicates the average length of the output sentences. “Avg. expr. Len.” indicates the average length of the input logic expressions. “Avg. Logic. Num.” indicates the average number of the logic operators in each logic expressions.

Origin:	Expression: exists x01 . (man (x01) & exists x02 . (bread (x02) & exists e03 . (make (e03) & (subj (e03) = x01) & (acc (e04) = x02)))) Meaning: A man is making bread
Add noisy variables:	Expression: exists x01 . (man (x01) & exists x02 . (dinner (x02) & exists x03 . (bread (x03) & exists e04 . (make (e04) & (subj (e04) = x01) & (acc (e04) = x03))))) Meaning: A man is making bread
Change subject and accusative:	Expression: exists x01 . (man (x01) & exists x02 . (bread (x02) & exists e03 . (make (e03) & (subj (e03) = x02) & (acc (e03) = x01)))) Meaning: A bread is making man

Fig. 3. The examples of two types of out-of-distribution samples.

found 4 incorrect samples. This result ensures that our dataset provides a reasonably high-quality benchmark to validate our model performance.

2) *Out-of-Distribution Set:* In addition to the in-distribution set, we also construct a challenging set to evaluate the models’ robustness to out-of-distribution samples. Our basic idea is that the content generated by the model should conform to the constraints of a logical expression, regardless of whether the expression conforms to common sense. Therefore, we constructed some counterfactual logical expressions. If the model can generate counterfactual sentences that meet the constraints of the logical expression, it can prove that the model truly understands the logical expression rather than just generating sentences that conform to common sense. Specifically, we make the following two modifications to the regular logic expressions:

- We add some noisy variables in logic expressions. For example in Fig. 3, we add a noisy variable “dinner,” and the meaning of the logic expression remains the same. If the models really learn how to generate sentences base on the logic expressions, they should ignore the noisy variable. Since “dinner” has no connection with other variables or events, it should not appear in the output sentence.
- We exchange the subject and accusative in the regular logic expressions. For example in Fig. 3, we exchange subject

and accusative in the last expressions. The meaning of the logic expression changes to “A bread is making man,” which is weird and counterfactual. However, if the models really learn how to generate sentences base on the logic expressions, they should generate such weird sentence.

We manually construct 1,620 out-of-distribution samples, of which 540 samples are use as the test samples and 1,080 samples are used as additional training data to fine-tune the models.

IV. MODEL

To alleviate the difficulty of capturing dependencies in sequential form, we propose SLEtoNL to generate sentences from logic expressions. Our model represents logic expressions with hierarchical structures, and use structural encoders to capture structural information in logic expressions. The overall architecture is illustrated in Fig. 4. Specifically, there are two phases in our model. In conversion phase, a logic expression is converted to two types of structural representations (i.e., tree or graph). For tree structure, we converts a sequential logic expression into a tree structure by a recursive parsing algorithm. For graph structure, we merge the duplicate nodes in a tree then convert it into a graph structure. In generation phase, to make the full use the structural representations, two types of structural encoder are used to capture the dependencies between nodes: a Tree-LSTM, which consists of a bottom up Tree-LSTM and a top down Tree-LSTM; a Graph Convolutional Network is used as the encoder for the graph structure input. Finally, a sequential decoder with the copy mechanism is applied to generate sentences. Additionally, we use a stacking encoder strategy introduced in [44] to enhance the tree and graph encoders.

A. Conversion Phase

1) *Tree Structure:* We propose a recursive algorithm to convert a logic expression into a tree structure. We first use several regular expressions to split logic expressions into tokens, with their orders in the original expressions preserved. We consider that all tokens between each “(” and “)” belong to the same subtree. We then detect every “(” and “)” and use tokens between “(” and “)” as input to construct subtrees. Finally, we collect all the subtrees and obtain a full tree structure, as shown in Fig. 5.

However, these trees merely preserve the coarse-grained structural information in the logic expression, and neglects the fine-grained structural information. For example in Fig. 5,

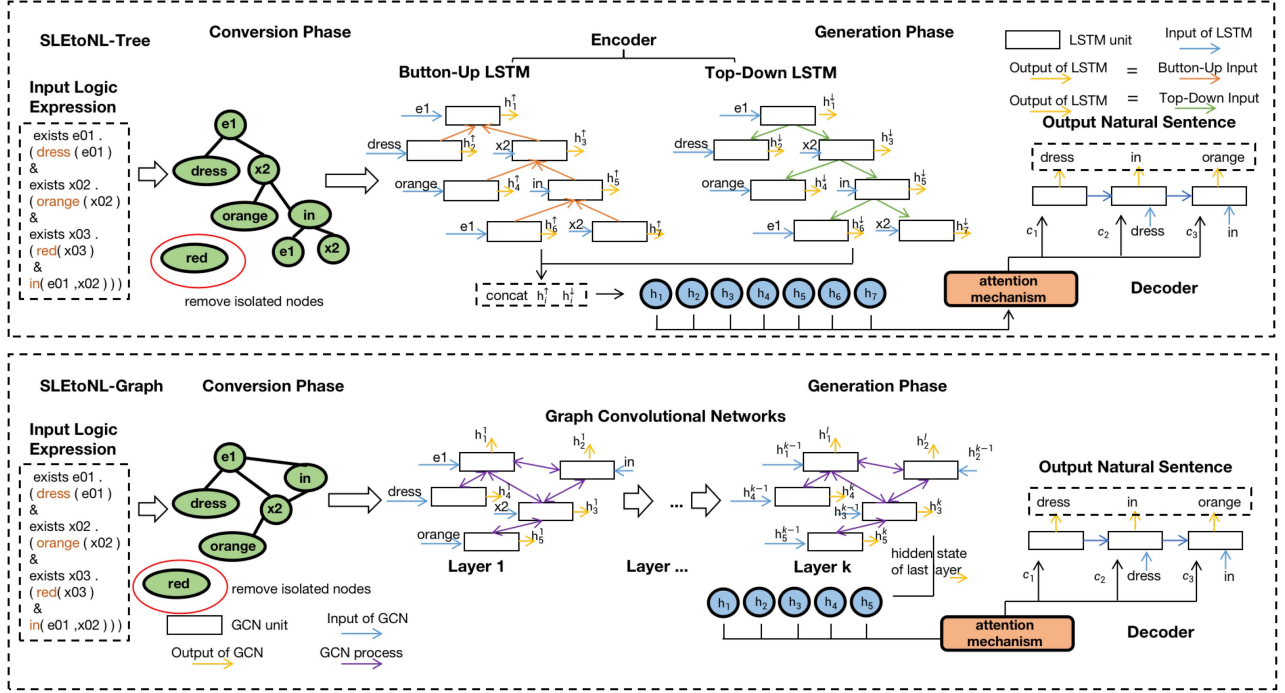


Fig. 4. The overall architecture of SLEtoNL-Tree (top) and SLEtoNL-Graph (below). Since the tree and graph are quite large, we simplify the structure of the tree and graph in the example.

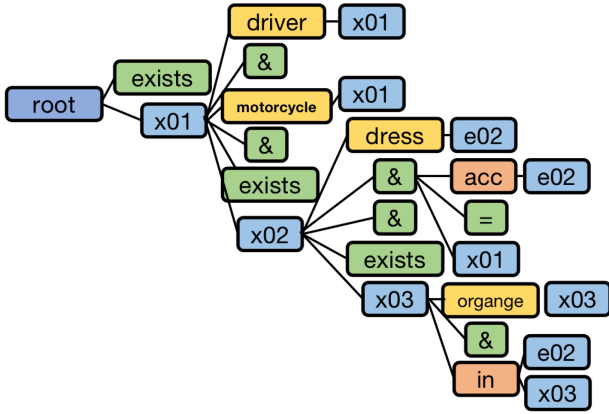


Fig. 5. The coarse-grained tree structure of the sentence "a motorcycle driver dressed in orange". We mark nodes with different colors. For nodes that describe logic relation (e.g., "exists," "&"), we mark them with green color. For nodes that describe entity or event (e.g., "x01," "e02"), we mark them with blue color. For nodes that describe attributes (e.g., "biker," "motorcycle"), we mark them with yellow color. For nodes that describe referring information (e.g., "subj," "in"), we mark them with orange color.

"driver" and "motorcycle" are connected by "&" in the logic expression, which demonstrates their relationship is "and". However in the tree structure (Fig. 5), "driver," "motorcycle" and "&" are at the same level. It is not intuitive that "&" is to describe the relationship between "driver" and "motorcycle". To address this issue, we introduce a fine-grained post-processing strategy. We revisit each subtree and detects nodes including "exists," "&," "|," "-", and "->". For "exists" node, its next right sibling subtree will move to its children nodes. For "&," "|," "-", and "->" node, their left and right sibling subtree will move to its children node.

After this post-processing procedure, the relationships between nodes become clearer, as described in Fig. 6.

2) *Graph Structure*: Duplicate nodes can appear in the tree structure. For example, in Fig. 6, there are three "x01" and four "e02". Since these duplicate nodes describe the same entity or event, we propose to merge them into one node to reduce the number of nodes. This results in a graph structure of logic expressions as describe in Fig. 7.

3) *Structure Refinement*: In out-of-distribution setting, there may be some noisy variables which have no connection to others. We simply check whether each nodes have at least one edge connected to others in the tree or graph structure, and then remove those nodes which have zero edge (i.e., isolated nodes).

B. Generation Phase

1) Encoders:

a) *Sequential encoder*: The sequential encoder is a bi-directional LSTM applied on the sequence input. A bi-directional LSTM consists of a forward LSTM and a backward LSTM. At each step i , forward hidden state $\vec{h}_i$ and backward hidden state $\overleftarrow{h}_i$ are concatenated to form a hidden state h_i of the encoder:

$$\vec{h}_i = \overrightarrow{\text{LSTM}}(x_i, \vec{h}_{i-1}), \quad (1)$$

$$\overleftarrow{h}_i = \overleftarrow{\text{LSTM}}(x_i, \overleftarrow{h}_{i-1}), \quad (2)$$

$$h_i = [\vec{h}_i, \overleftarrow{h}_i], \quad (3)$$

where x_i is the input. We call the model seq2seq in the experiments.

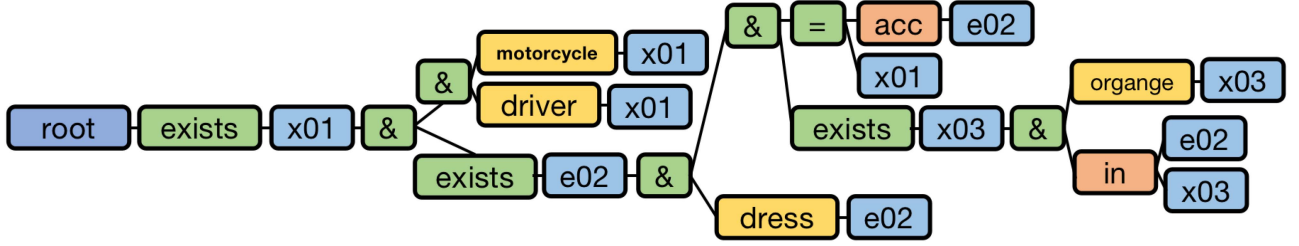


Fig. 6. The fine-grained tree structure of the sentence “a motorcycle driver dressed in orange”.

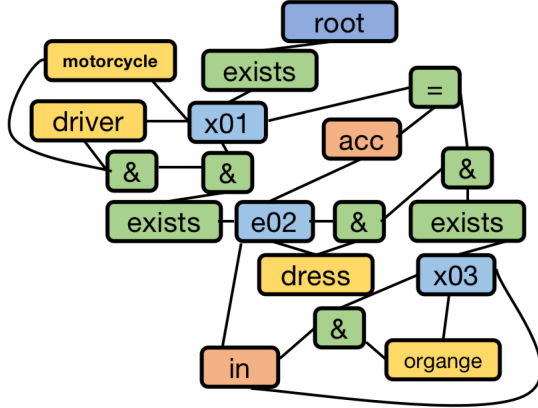


Fig. 7. The graph structure of the sentence “a motorcycle driver dressed in orange”.

b) Tree encoder: We use a Tree-Structured LSTM [45] as the tree encoder. Tree encoder take the tree representation of the logic expression as input. We use an implementation Tree-LSTM in [44], which consists of a bottom up Tree-LSTM and a top down Tree-LSTM. The bottom up Tree-LSTM is a Child-Sum Tree-LSTM. For each node v_i in the tree, the bottom up hidden states $h_i^\uparrow$ is updated based on its children $C(i)$. All the hidden states of its children are summed up in a single vector $\tilde{h}_i^\uparrow$, which is then passed into recurrent gates:

$$\tilde{h}_i^\uparrow = \sum_{k \in C(i)} h_k^\uparrow, \quad (4)$$

$$h_i^\uparrow = \text{LSTM}^\uparrow(\tilde{h}_i^\uparrow, x_i), \quad (5)$$

where x_i is the input. The top-down state of the root node is obtained by feeding the bottom-up state of the root node through a feed-forward layer:

$$h_{root}^\downarrow = \tanh(W_1 h_{root}^\uparrow + b_1), \quad (6)$$

where W_1 and b_1 are trainable parameters. The top down hidden states for all other nodes are computed with an another LSTM:

$$h_i^\downarrow = \text{LSTM}^\downarrow(h_{p(i)}^\downarrow, x_i), \quad (7)$$

where $p(i)$ is the parent of node v_i and x_i is the input. We concatenated the bottom up hidden states and the up down

hidden states as the final final hidden states:

$$h_i = [h_i^\uparrow, h_i^\downarrow]. \quad (8)$$

We call the model as SLEtoNL-Tree in this article.

c) Graph encoder: The graph encoder generates a node embedding for each node by accumulating information from its neighbors, and produces a graph embedding for the entire graph by abstracting all node embeddings. We use the traditional, non-recurrent, implementation of Graph Convolutional Network (GCN) [44] as our graph encoder. The node embeddings are computed as:

$$h_i^{k+1} = \sigma \left(\sum_{j \in N(i)} W_{dir(j,i)}^k h_j^k + b_2^k \right), \quad (9)$$

where h_i^k is the node embedding of node v_i at layer k , σ is a non-linear activation function, $N(i)$ is the set of the immediate neighbors of v_i , $W_{dir(j,i)}$ and b_2 are the trainable parameters. We use the the hidden vectors from the last layer of the GCN network to represent the final hidden states. We call this model SLEtoNL-Graph.

d) Stacking encoders: Previous studies have shown that stacking the Tree-LSTM and GCN with the sequential encoder provide improved performance in transformation for structural data to natural sentences [46], [47]. There have two variants of the stacking encoders, including *Structure on Top of Sequence* where GCN or Tree-LSTM is applied on the output of the BiLSTM, and *Sequence on Top of Structure* where the BiLSTM is applied on the output of GCN or Tree-LSTM. We use the latter variant due to its better performance shown in previous studies in [44]. We first apply the structural encoder to the structural input, and the GCN or Tree-LSTM representations are then fed into the BiLSTM. We call the two models SLEtoNL-TreeSeq and SLEtoNL-GraphSeq respectively.

2) Decoder: Given the hidden states from the encoders, we use a one-layer uni-directional LSTM with attention as the decoder to generates the corresponding natural language:

$$s_t = \text{LSTM}(y_{t-1}, s_{t-1}, c_t), \quad (10)$$

$$P(y_t | y_{<t}, X) = \text{Softmax}(W_o s_t), \quad (11)$$

where y_{t-1} is the output token of previous time step, s_{t-1} is the hidden state of previous time step and c_t is the context vector of current time. Each hidden state of decoder s_t is then linearly projected with a trainable weight matrix W_o and passed through a *softmax* layer to compute the probability of output

y_t . The context vector c_t dynamically computed using attention mechanism over the hidden states $(h_1, h_2, \dots, h_V)$ of encoders.

a) *Copy mechanism*: We adopt the copy mechanism [48], [49] to solve the unknown word problem, such as dates, numbers and named entities. The copy mechanism works on top of an attention-based decoder by integrating the attention distribution into the final vocabulary distribution. The final probability distribution is defined as the interpolation between two probability distributions:

$$P_{\text{final}} = \theta_t P_{\text{vocab}} + (1 - \theta_t) P_{\text{attn}}, \quad (12)$$

where θ_t is a switch for controlling generating a word from the vocabulary or directly copying it from the input representation. P_{vocab} is the vocabulary probability distribution as defined in (11), and P_{attn} is calculated based on the current attention distribution by merging probabilities of duplicated words. Finally, θ_t is defined as:

$$\theta_t = \sigma(W_s s_t + b_3), \quad (13)$$

where w_s^T and b_3 is the model parameters.

C. Training and Inference

Our method is trained to maximize the conditional log-probability of the reference natural language given a source logic expression. In inference, we use the beam search to generate sentences.

V. EXPERIMENTS

In this section, we explain the experimental details and then comprehensively report the automatic evaluation of different encoders. Finally, we conduct detailed human evaluation and error analysis.

A. Baselines

We test several common used natural generation models on our constructed dataset, including the seq2seq model and several pretrained natural language models. The hyperparameters are the default parameters in the fine-tuning script provided by SimpleTransformers.² We manually tuned their hyperparameters based on the dev set performance on BLEU.

a) *seq2seq-LSTM*: We employ the seq2seq model [8] as a baseline. We implement the seq2seq model by the OpenNMT-py³ library. We employ a 2-layer LSTM [50] as an encoder and a 2-layer LSTM with the copy attention mechanism [48], [49] as a decoder.

b) *seq2seq-transformer*: We also employ the transformer-based [51] seq2seq model [8] as a baseline. We implement the seq2seq-Transformer model by the OpenNMT-py⁴ library. We use the hyperparameters provided by OpenNMT-py, where `enc_layers` is 6, `dec_layers` is 8, `heads_num` is 8. More details are shown in this script.⁵

c) *State-of-the-art NLG models*: We also employ several state-of-the-art pretrained language models as strong baselines, including GPT-3 [10], BART [11], T5 [12]. The input of these models is the sequential logic expressions, and the output is the corresponding natural language. We use all the training examples to train the models.

B. Experiment Setup

We evaluate the different generation models on the constructed dataset, including an in-distribution set which contains 9,000/1,000/1,000 samples for training/development/testing and an out-of-distribution set which contains 1,080/540 for training/testing. We use BLEU-1,2,3,4⁶ [52] and Meteor⁷ [53] as evaluation metrics. Our models are implemented in Pytorch [54] with OpenNMT-py [55]. For the GCN components, we use two layers, ReLU activations, and tanh highway layers. We use single layer LSTMs. We use SGD with the initial learning rate set to 1 and decay to 0.8. Batch size is set to 32. The computing infrastructure we used is an ubuntu server with Tesla P100.

C. Results

We first evaluate the overall performance of the different generation models on in-distribution test set. Table III shows the comparison results. Except the pretrained language models, we observe that our SLEtoNL models which based on non-sequential input achieves better performance than the sequential model. This illustrates that parsing and modeling the structure of the logic expression helps to generate better natural language. It also demonstrates the effectiveness of the structural encoders. We also see that the models based on tree input achieve better performance than the models based on graph input, which is not in line with our expectation. One possible reason is that in graph representation, one entity node may link several attribute nodes and referring nodes. As a result, it would be more difficult for the models to understand the true relationships between nodes. By contrast, in tree representation, the relation between nodes can be identified more easily. We make a further analysis in the case study below.

For SLEtoNL-Tree and SLEtoNL-Graph, we find that the stacking encoders approaches (SLEtoNL-TreeSeq and SLEtoNL-GraphSeq) achieve better scores, highlighting the importance of a sequential component. When the sequential encoder is removed, the performance of the SLEtoNL-Graph drops, even worse than the sequential encoder. On the other hand, the sequential encoder has less impact on SLEtoNL-Tree. These results align with the results in [44], the result is expected as the SLEtoNL-Tree already uses LSTM gates in the computation.

We also observe that all the pretrained language models (i.e., T5, BART, GPT3) achieve superior performance merely based on sequential input. A possible reason is that the expressions in most output sentences are quite common and might have semantically similar variants in the training data of pretrained

²[Online]. Available: <https://simpletransformers.ai/>

³[Online]. Available: <https://github.com/OpenNMT/OpenNMT-py>

⁴[Online]. Available: <https://github.com/OpenNMT/OpenNMT-py>

⁵[Online]. Available: <https://opennmt.net/OpenNMT-py/FAQ.html#how-do-i-use-the-transformer-model>

⁶[Online]. Available: www.nltk.org/_modules/nltk/translate/bleu_score.html

⁷[Online]. Available: www.nltk.org/_modules/nltk/translate/meteor_score.html

TABLE III
BLEU AND METEOR (%) SCORES ON THE IN-DISTRIBUTION TEST SET

Input Representation	Model	BLEU-1	BLEU-2	BLEU-3	BLEU-4	Meteor
Seq	seq2seq-LSTM	87.2	69	55.6	44.1	82.5
	seq2seq-Transformer	81.3	53.7	38	25	66.3
Tree	SLEtoNL-Tree	87.2	69.1	56.2	44.8	83.4
	SLEtoNL-TreeSeq	88.1	73.6	62.3	51.2	85.1
	SLEtoNL-Graph	84.8	66.8	54.2	42.4	82.1
	SLEtoNL-GraphSeq	90.6	75.8	64.4	53.9	85.5
Graph	SLEtoNL-Graph	88.0	68.5	55.3	43.2	81.0
	SLEtoNL-GraphSeq	90.1	74.7	63.2	52.6	83.7
Pretrained Language Models	T5	92.9	81.8	72.8	64.1	91.2
	BART	94.2	85.4	78.1	71.1	91.6
	GPT3	94.2	83.9	75.6	67.5	91.7

TABLE IV
THE RESULTS ON OUT-OF-DISTRIBUTION TEST SET WITHOUT ADDITIONAL TRAINING

Models	BLEU-1	BLEU-2	BLEU-3	Meteor
T5	60.5	25.4	7.7	30.7
BART	52.6	17.8	3.8	29.8
GPT3	52.8	14.7	3	27.8
SLEtoNL-TreeSeq (Ours)	63.0	33.4	10.3	31.9

TABLE V
THE RESULTS ON OUT-OF-DISTRIBUTION TEST SET WITH ADDITIONAL TRAINING

Models	BLEU-1	BLEU-2	BLEU-3	Meteor
T5	70.0	48.7	28.3	39.2
BART	73.2	52.9	31.9	38.2
GPT3	89.7	77.8	67.9	47.2
SLEtoNL-TreeSeq (Ours)	99.1	97.9	96.5	52.5

language models, and hence these pretrained models are likely to achieve good performance. This hypothesis is confirmed by the results on the out-of-distribution test set below.

The results on the out-of-distribution test set are shown in Table IV. We observe that all the models' performance drop drastically without additional training. This demonstrates that models trained on regular logic expressions cannot handle out-of-distribution expressions well. However, our SLEtoNL-tree achieves better performance than pretrained language models. This demonstrates the effectiveness of the structural representation of logic expressions. Our model can better capture the relationships between variables based on the structured representation and thus generate sentences that match the semantics of the logical expressions. In contrast, the pre-trained models only learn how to combine a common sentence with the words that appear in the logical expression, and hence cannot perform well in out-of-distribution setting.

To improve the performance on the out-of-distribution set, one possible method is to use adversarial examples to fine tune the models. We using the 1,080 out-of-distribution examples as the additional training data to fine tune the models. The results are shown in Table V. We observe that our SLEtoNL-tree achieves decent performance improvement on out-of-distribution test set after training on adversarial examples. On the contrary, although the performance of the pre-trained language models also improve, there is still a large gap between their performance

TABLE VI
THE HUMAN EVALUATION RESULTS OF "FLUENCY" OF SLEtoNL-GRAPHSEQ ON DIFFERENCE INPUT TYPES

Model	not fluent	partial-fluent	fluent
Seq2Seq-LSTM	1	26 / 49	73
(Ours)tree input	2	20 / 50	78
(Ours)graph input	1	21 / 49	79

The number in partial-fluent represents (most not fluent / most fluent).

on the in-distribution test set. This shows the effectiveness and importance of structured representations in dealing with out-of-distribution samples. In the process of structuring logical expressions, noise variables can be removed to reduce the impact of noisy data. At the same time, different relationships between variables will be presented through different structures, which is more conducive to the model to capture the corresponding features to generate correct natural language.

D. Human Evaluation

The automatic evaluation cannot fully reflect the quality of the generated text. We further investigate the quality of the generated text by human evaluation. From our initial observation, we find that the best models can generate exactly the correct text for most short sentences. Thus we focus on investigating the long sentences in human evaluation. Specifically, we sample 60 sentences containing more than 50 characters from the output of Seq2Seq-LSTM, SLEtoNL-GraphSeq (tree input) and SLEtoNL-GraphSeq(graph input), and check their quality by 2/3 human annotators (graduate students from the software engineering school) independently. Specifically, the annotators are asked to score the generated text in terms of two aspects: 1) fluency: the fluency evaluation contains three categories: not fluent, partial-fluent (most fluent or most not fluent), and fluent. 2) logicity: the logicity evaluation contains three categories: wrong, partial-correct (most correct or most not correct), and correct. Note that if the generated text and the reference text are logically equal, it should be scored as correct. For example, "two men and one woman wear black jacket" are logically equal to "two men wear black jacket and one woman wear black jacket"

We show the results in Tables VI and VII, from which we observe that human evaluation results are in line with the automatic results. Moreover, we can further observe that the tree

TABLE VII
THE HUMAN EVALUATION RESULTS OF “LOGICALITY” OF
SLEtoNL-GRAPHSeq ON DIFFERENCE INPUT TYPES

Model	wrong	partial-correct	correct
Seq2Seq-LSTM	11	51 / 65	22
Ours(tree input)	11	48 / 62	29
Ours(graph input)	10	49 / 71	20

The number in partial-correct represent (most not correct/most correct).

representation of the logic expression can improve the logicality of the generated sentence. For the fluency, except the poor performance of the Seq2Seq-LSTM, there is not much difference between the two SLEtoNL models. However, both the two SLEtoNL models do not achieve decent performance in human evaluation (most generated sentences are scored as partial-correct). These results demonstrate the challenges for generating natural sentences from logic expressions.

E. Case Study

1) *In-Distribution Cases*: Table VIII shows the cases study which comparing Seq2Seq-LSTM and our SLEtoNL-TreeSeq. Although the Seq2Seq-LSTM can generate most words correctly, some critical words are omitted or misplaced by Seq2Seq-LSTM. In the first example, Seq2Seq-LSTM omits the word “bikini” in the logic expression, and misplaces it with “volleyball”. In the second example, Seq2Seq-LSTM omits the “standing” which describes the activity of the subject. In contrast, with the help of structural representation, SLEtoNL can capture all the words and their dependencies, thereby generating correct sentences.

2) *Out-of-Distribution Cases*: We first examine the following case with noisy words. Given the logic expression: “exists x01. (man (x01) & exists x02. (bread (x02) & exists x03. (dinner (x03) & exists e04. (make (e04) & (subj (e04) = x01) & (acc (e04) = x02))))), “ the corresponding sentence is “a man is making bread”. BART, T5, GPT3 output a sentence “a man is making bread for dinner,” and cannot ignore the noisy variable “dinner”. In contrast, our model outputs the correct sentence “a man is making bread”. This illustrates that our model are more robust to the noise by eliminating an isolated word node, i.e., the noisy word “dinner” in this case.

We then examine the subject and accusative exchange case. Given the logic expression: “exists x01. (dinner (x01) & exists x02. (man (x02) & exists x03. (bread (x03) & exists e04. (make (e04) & (subj (e04) = x01) & (acc (e04) = x03))))). The corresponding sentence is “a dinner is making bread”. BART, T5, GPT3 output a sentence “dinner that a man makes bread”. These models use all the words in the logic expression to form a normal sentence but not match the corresponding semantic of the logic expression. In contrast, our model can output the correct sentence “dinner is making bread”.

VI. DISCUSSION

A. Tree or Graph Representation

Table III shows that SLEtoNL based on tree input achieve better performance than the models based on graph input, which

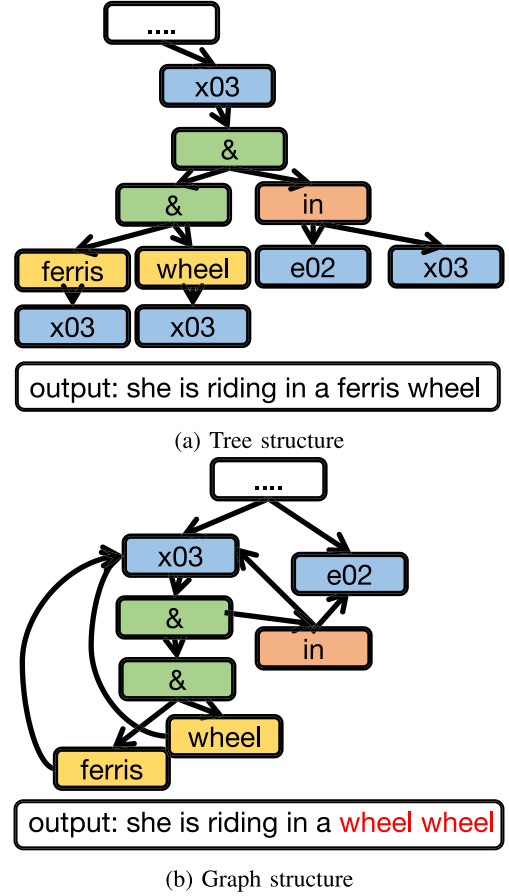


Fig. 8. The tree and graph structure of sentence “she is riding in a ferris wheel.”

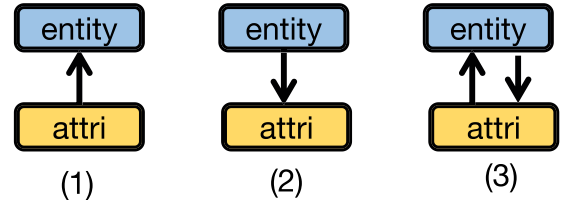


Fig. 9. Three different types of graph structure.

is not in line with our expectation. To further explore the differences between the models, we performed a case study of the models’ output. Fig. 8(a) and (b) show a representative example, where we can see that the SLEtoNL-Graph outputs a wrong sentence, while the output of the SLEtoNL-Tree is correct. One possible reason is that the node “x03” in the graph is linked directly with the node “ferris” and the node “wheel,” and all their features are fed into the node “x03”’s representation. As a result, the model cannot handle these features and thus output a repeated “wheel”.

To overcome this issue, we further conduct several comparison experiments: (1) baseline: using the graph representation described in Section IV-A2, and using the SLEtoNL-GraphSeq. (2) variant-1: Compared with baseline, using the reverse direction edge. (3) variant-2: compared with baseline, using the bi-directional edge. The examples are shown in Fig. 9. The BLEU-4 results are shown in Table IX. We can see that, variant-1

TABLE VIII
TWO EXAMPLES OF THE SENTENCES GENERATED BY SEQ2SEQ-LSTM AND SLEtoNL-TREESEQ

Input Logic Expressions	Output Natural Sentence
exists x01.(woman(x01) & exists x02.(bikini(x02) & blue(x02) & white(x02) & exists x03.(volleyball(x03) & exists e04.(playing(e04) & (subj(e04)=x02) & (acc(e04)=x03))) & exists e05.(in(e05, x02) & (subj(e05)=x01))))	REF: a woman in a blue and white bikini playing volleyball Seq2Seq-LSTM: a woman in a blue and white volleyball playing volleyball SLEtoNL-TreeSeq: a woman in a blue and white bikini playing volleyball
exists x01.(man(x01) & exists x02.(brown(x02) & exists e03.(standing(e03) & (subj(e03)=x02) & exists x04.(front(x04) & exists x05.(building(x05) & exists x06.(cooler(x06) & exists e07.(with(e07, x06) & (subj(e07)=x05))) & (x04=x05))) & in_front_of(e03, x04))) & exists e08.(in(e08, x02) & (subj(e08)=x01))))	REF: a man in brown standing in front of a building with coolers Seq2Seq-LSTM: a man in brown in front of a building with cooler SLEtoNL-TreeSeq: a man in brown standing in front of a building with coolers

REF is the reference sentence.

TABLE IX
THE RESULTS OF DIFFERENT GRAPH STRUCTURES

Input	Graph		
Model	baseline	variant-1	variant-2
BLEU-4	52.6	53.8	49.1

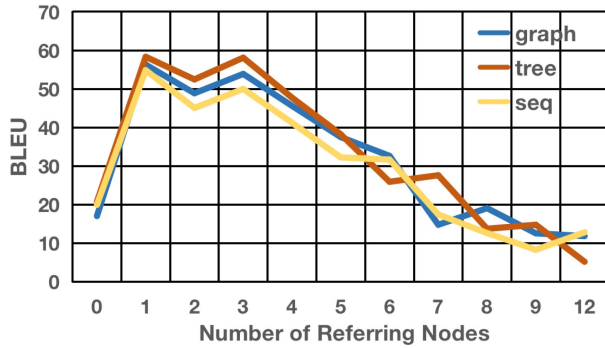


Fig. 10. The BLEU-4 scores of the models with different referring nodes.

and variant-2 can achieve better performance than baseline. It demonstrates that the incoming edges of the entity nodes would harm the performance (baseline and variant-2). When changing the incoming edges into outgoing edges (variant-1), the BLEU-4 score increases, however, still lower than the tree model. The results suggest that the tree representation is more suitable than graph.

B. Number of Referring Nodes

Since we propose to use tree or graph models to capture the complex relations in logic expression, we expect them to deal better with the structural information. Although the overall scores show an advantage of the structural encoders, it is still not clear how this is achieved. We expect that the benefit of the structural encoders will be evident for those examples containing more complex structures. The number of referring nodes (i.e., “subj,” “acc”) is an important feature of whether a graph is complex. If a graph contains more referring nodes, it would be more difficult for the models to understand the relationships between nodes.

We evaluate the influence of the number of referring nodes on the model performance. Fig. 10 shows the BLEU-4 scores of

TABLE X
COMPARING RESULTS WITH AMR REPRESENTATIONS

Model	BLEU4	Meteor
SLEtoNL-Tree	44.8	83.4
SLEtoNL-Graph	43.2	81.0
AMR-Graph2seq	34.1	66.3

different referring nodes. It can be observed that the structural models can outperform the sequential model in most cases. The widest gap between the structural models and the sequential model is when the example contains 1 to 5 referring nodes. However, the structural models do not achieve a decent improvement when the number of referring nodes is more than 6, it would be our future work to improve the model to handle a more complex logic structure.

C. Compared With AMR Representation

In addition to logical expressions, there are other formal languages used to express the semantics of sentences, such as the Abstract Meaning Representation (AMR) [24]. The reason why we choose to use logical expressions instead of AMR is because logical expressions can be further used for logical reasoning, while AMR is not suitable for inference [24]. Although in this article we focus on how to generate natural language from logical expressions, an interesting question is that what is the difference if we use the AMR to represent our constructed dataset.

For AMR, we use the amrlib⁸ to parse the 12,000 sentences into AMRs. Then we use the sentences and their AMRs to train a graph2seq model. The results are shown in Table X. Compared with models based on the AMR, our model that based on the logic expressions representation can achieve a better performance on all metrics. One possible reason is that the testing dataset are constructed from the natural language inference dataset, while logical expressions are more suitable for representing this type of data.

VII. CONCLUSION

In this article, we investigate how to generate natural language from a given logic expression. We propose a model to convert Structural Logic Expressions into Natural Language

⁸[Online]. Available: <https://github.com/bjascob/amrlib>

(SLEtoNL). Specifically, SLEtoNL converts a logic expression into a tree or graph structure and then generates sentences based on structural representation. We introduce a parallel dataset for evaluation, where each sample contains a natural language sentence and its corresponding logic expression. Compared with the sequential model, our method can capture structural information in logic expressions. The quantitative and qualitative analysis suggest that the tree structure is more suitable for this generation task. And our model outperforms pretrained language models (e.g., T5, GPT3) with a large margin (28.6 in BLEU3) in out-of-distribution evaluation.

ACKNOWLEDGEMENT

The authors sincerely thank Dr. Junying Chen from the School of Software Engineering at the South China University of Technology for in-depth discussions and valuable suggestions.

REFERENCES

- [1] L. Wos, R. Overbeek, E. Lusk, and J. Boyle, *Automated Reasoning*. Hoboken, NJ, USA: Prentice Hall, 1984.
- [2] K. R. Apt, "Logic programming," in *Handbook of Theoretical Computer Science, Volume B: Formal Models and Semantics (B)*, vol. 1990. Cambridge, MA, USA: MIT Press, 1990.
- [3] V. D'silva, D. Kroening, and G. Weissenbacher, "A survey of automated techniques for formal software verification," *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, vol. 27, no. 7, pp. 1165–1178, Jul. 2008.
- [4] C. Kern and M. R. Greenstreet, "Formal verification in hardware design: A survey," *ACM Trans. Des. Automat. Electron. Syst.*, vol. 4, no. 2, pp. 123–193, 1999.
- [5] P. Martínez-Gómez, K. Mineshima, Y. Miyao, and D. Bekki, "Ccg2lambda: A compositional semantics system," in *Proc. Assoc. Comput. Linguistics Syst. Demonstrations*, 2016, pp. 85–90.
- [6] R. Suzuki, H. Yanaka, M. Yoshikawa, K. Mineshima, and D. Bekki, "Multimodal logical inference system for visual-textual entailment," in *Proc. 57th Annu. Meeting Assoc. Comput. Linguistics: Student Res. Workshop*, 2019, pp. 386–392.
- [7] F. Arabshahi, J. Lee, M. Gawarecki, K. Mazaitis, A. Azaria, and T. Mitchell, "Conversational neuro-symbolic commonsense reasoning," in *Proc. AAAI Conf. Artif. Intell.*, 2021, pp. 4902–4911.
- [8] D. Bahdanau, K. H. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," in *Proc. 3rd Int. Conf. Learn. Representations*, 2015.
- [9] I. Sutskever, O. Vinyals, and Q. V. Le, "Sequence to sequence learning with neural networks," in *Proc. 27th Int. Conf. Neural Inf. Process. Syst.*, 2014, pp. 3104–3112.
- [10] T. Brown et al., "Language models are few-shot learners," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 1877–1901.
- [11] M. Lewis et al., "Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension," in *Proc. 58th Annu. Meeting Assoc. Comput. Linguistics*, 2020, pp. 7871–7880.
- [12] C. Raffel et al., "Exploring the limits of transfer learning with a unified text-to-text transformer," *J. Mach. Learn. Res.*, vol. 21, pp. 5485–5551, 2020.
- [13] K. Kukich, "Design of a knowledge-based report generator," in *Proc. 21st Annu. Meeting Assoc. Comput. Linguistics*, 1983, pp. 145–150.
- [14] D. D. McDonald and J. Pustejovsky, "A computational theory of prose style for natural language generation," in *Proc. 2nd Conf. Eur. Chapter Assoc. Comput. Linguistics*, 1985, pp. 187–193.
- [15] E. Reiter and R. Dale, "Building applied natural language generation systems," *Natural Lang. Eng.*, vol. 3, no. 1, pp. 57–87, 1997.
- [16] X. Zhao, W. Wu, C. Xu, C. Tao, D. Zhao, and R. Yan, "Knowledge-grounded dialogue generation with pre-trained language models," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2020, pp. 3377–3390.
- [17] Y. Su, W. Yan, D. Cai, S. Baker, A. Korhonen, and N. Collier, "Prototype-to-style: Dialogue generation with style-aware editing on retrieval memory," *IEEE/ACM Trans. Audio, Speech Lang. Process.*, vol. 29, pp. 2152–2161, 2021.
- [18] Q. Huang et al., "Story ending generation with multi-level graph convolutional networks over dependency trees," in *Proc. AAAI Conf. Artif. Intell.*, 2021, pp. 13073–13081.
- [19] G. Chen, Y. Liu, H. Luan, M. Zhang, Q. Liu, and M. Sun, "Learning to generate explainable plots for neural story generation," *IEEE/ACM Trans. Audio, Speech Lang. Process.*, vol. 29, pp. 585–593, 2021.
- [20] Y. Zheng, R. Zhang, M. Huang, and X. Mao, "A pre-training based personalized dialogue generation model with persona-sparse data," in *Proc. AAAI Conf. Artif. Intell.*, 2020, pp. 9693–9700.
- [21] S. Dathathri et al., "Plug and play language models: A simple approach to controlled text generation," in *Proc. Int. Conf. Learn. Representations*, 2019.
- [22] Z. Chen et al., "Logic2text: High-fidelity natural language generation from logical forms," in *Proc. Conf. Empirical Methods Natural Lang. Process.: Findings*, 2020, pp. 2096–2111.
- [23] W. Chen, J. Chen, Y. Su, Z. Chen, and W. Y. Wang, "Logical natural language generation from open-domain tables," in *Proc. 58th Annu. Meeting Assoc. Comput. Linguistics*, 2020, pp. 7929–7942.
- [24] L. Banarescu et al., "Abstract meaning representation for sembanking," in *Proc. 7th Linguistic Annotation Workshop Interoperability Discourse*, 2013, pp. 178–186.
- [25] J. Flanagan, C. Dyer, N. A. Smith, and J. G. Carbonell, "Generation from abstract meaning representation using tree transducers," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics: Hum. Lang. Technol.*, 2016, pp. 731–739.
- [26] L. Song, X. Peng, Y. Zhang, Z. Wang, and D. Gildea, "AMR-to-text generation with synchronous node replacement grammar," in *Proc. 55th Annu. Meeting Assoc. Comput. Linguistics*, 2017, pp. 7–13.
- [27] G. Lampouras and A. Vlachos, "Sheffield at SemEval-2017 Task 9: Transition-based language generation from AMR," in *Proc. 11th Int. Workshop Semantic Eval.*, 2017, pp. 586–591.
- [28] N. Gruzitis, D. Gosko, and G. Barzdins, "RIGOTRIO at SemEval-2017 Task 9: Combining machine learning and grammar engineering for AMR parsing and generation," in *Proc. 11th Int. Workshop Eval.*, 2017, pp. 924–928.
- [29] I. Konstas, S. Iyer, M. Yatskar, Y. Choi, and L. Zettlemoyer, "Neural AMR: Sequence-to-sequence models for parsing and generation," in *Proc. 55th Annu. Meeting Assoc. Comput. Linguistics*, 2017, pp. 146–157.
- [30] S. Takase, J. Suzuki, N. Okazaki, T. Hirao, and M. Nagata, "Neural headline generation on abstract meaning representation," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2016, pp. 1054–1059.
- [31] L. Song, Y. Zhang, Z. Wang, and D. Gildea, "A graph-to-sequence model for AMR-to-text generation," in *Proc. 56th Annu. Meeting Assoc. Comput. Linguistics*, 2018, pp. 1616–1626.
- [32] D. Cai and W. Lam, "Graph transformer for graph-to-sequence learning," in *Proc. AAAI Conf. Artif. Intell.*, 2020, pp. 7464–7471.
- [33] J. Bos, "Expressive power of abstract meaning representations," *Comput. Linguistics*, vol. 42, no. 3, pp. 527–535, 2016.
- [34] E. Stabler, "Reforming AMR," in *Proc. Formal Grammar: 22nd Int. Conf.*, 2018, pp. 72–87.
- [35] R. Blanck and A. Maskharashvili, "From tree adjoining grammars to higher order representations of abstract meaning representations via abstract categorical grammars," in *Logic and Algorithms in Computational Linguistics*. Berlin, Germany: Springer, 2020, pp. 67–93.
- [36] N. Schrack, R. Cui, H. López, and D. Hershovich, "Can AMR assist legal and logical reasoning," in *Proc. Findings Assoc. Comput. Linguistics: EMNLP*, 2022, pp. 1555–1568.
- [37] P. Kapanipathi et al., "Leveraging abstract meaning representation for knowledge base question answering," in *Proc. Findings Assoc. Comput. Linguistics: ACL-IJCNLP*, 2021, pp. 3884–3894.
- [38] M. J. Gordon and T. F. Melham, *Introduction to HOL: A Theorem Proving Environment for Higher Order Logic*. Cambridge, U.K.: Cambridge Univ. Press, 1993.
- [39] K. Mineshima, P. Martínez-Gómez, Y. Miyao, and D. Bekki, "Higher-order logical inference with compositional semantics," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2015, pp. 2055–2061.
- [40] J. Rao, K. Upasani, A. Balakrishnan, M. White, A. Kumar, and R. Subba, "A tree-to-sequence model for neural NLG in task-oriented dialog," in *Proc. 12th Int. Conf. Natural Lang. Gener.*, 2019, pp. 95–100.
- [41] K. Xu, L. Wu, Z. Wang, Y. Feng, and V. Sheinin, "SQL-to-text generation with graph-to-sequence model," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2018, pp. 931–936.
- [42] S. Bowman, G. Angeli, C. Potts, and C. D. Manning, "A large annotated corpus for learning natural language inference," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2015, pp. 632–642.

- [43] S. Clark and J. R. Curran, "Parsing the WSJ using CCG and log-linear models," in *Proc. 42nd Annu. Meeting Assoc. Comput. Linguistics*, 2004, pp. 103–110.
- [44] M. Damonte and S. B. Cohen, "Structural neural encoders for AMR-to-text generation," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics: Hum. Lang. Technol.*, 2019, pp. 3649–3658.
- [45] K. S. Tai, R. Socher, and C. D. Manning, "Improved semantic representations from tree-structured long short-term memory networks," in *Proc. 53rd Annu. Meeting Assoc. Comput. Linguistics 7th Int. Joint Conf. Natural Lang. Process.*, 2015, pp. 1556–1566.
- [46] A. Eriguchi, K. Hashimoto, and Y. Tsuruoka, "Tree-to-sequence attentional neural machine translation," in *Proc. 54th Annu. Meeting Assoc. Comput. Linguistics*, 2016, pp. 823–833.
- [47] D. Marcheggiani and I. Titov, "Encoding sentences with graph convolutional networks for semantic role labeling," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2017, pp. 1506–1515.
- [48] C. Gulcehre, S. Ahn, R. Nallapati, B. Zhou, and Y. Bengio, "Pointing the unknown words," in *Proc. 54th Annu. Meeting Assoc. Comput. Linguistics*, 2016, pp. 140–149.
- [49] J. Gu, Z. Lu, H. Li, and V. O. Li, "Incorporating copying mechanism in sequence-to-sequence learning," in *Proc. 54th Annu. Meeting Assoc. Comput. Linguistics*, 2016, pp. 1631–1640.
- [50] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural computation*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997.
- [51] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. neural Inf. Process. Syst.*, 2017, pp. 5998–6008.
- [52] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "BLEU: A method for automatic evaluation of machine translation," in *Proc. 40th Annu. Meeting Assoc. Comput. Linguistics*, 2002, pp. 311–318.
- [53] S. Banerjee and A. Lavie, "METEOR: An automatic metric for MT evaluation with improved correlation with human judgments," in *Proc. ACL Workshop Intrinsic Extrinsic Eval. Measures Mach. Transl. Summarization*, 2005, pp. 65–72.
- [54] A. Paszke et al., "Automatic differentiation in pytorch," in *Proc. Conf. Neural Inf. Process. Syst. Autodiff Workshop*, 2017, pp. 1–4.
- [55] G. Klein, Y. Kim, Y. Deng, J. Senellart, and A. M. Rush, "OpenNMT: Open-source toolkit for neural machine translation," in *Proc. ACL Syst. Demonstrations*, 2017, pp. 67–72.



Yi Cai (Member, IEEE) received the Ph.D. degree in computer science from The Chinese University of Hong Kong, Hong Kong. He is currently a Professor with the South China University of Technology, Guangzhou, China. His research interests include recommendation systems, personalized search, semantic webs, and data mining. His research works are published in many conferences and journals, such as IEEE TRANSACTIONS ON KNOWLEDGE AND DATA ENGINEERING, *Neural Networks*, *Knowledge-based Systems*, *Engineering Applications of Artificial Intelligence*, and *Neurocomputing*, and AAAI, COLING, CIKM, AAMAS, DAS-FAA, and other international conferences about perspective mining, cognitive modeling, information retrieval, and semantic analysis.



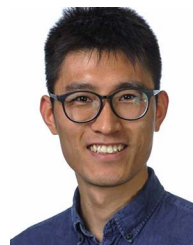
Zetao Lian is currently working toward the B.S. degree with the School of Software Engineering, South China University of Technology, Guangdong, China. His research interests include natural language processing and knowledge graph.



Ho-fung Leung (Senior Member, IEEE) received the B.Sc. and M.Phil. degrees in computer science from The Chinese University of Hong Kong, Hong Kong, and the Ph.D. degree from the University of London, London, U.K., with DIC (Diploma of Imperial College) in computing from Imperial College London, London. He is currently a Professor with the Department of Computer Science and Engineering, The Chinese University of Hong Kong, Hong Kong. He has authored more than 250 publications, including five research monographs, and five edited volumes. In 1998, he was the Chairperson of ACM (Hong Kong Chapter). He is a Chartered Fellow of the BCS, Fellow of the HKIE, and Full Member the HKCS. He is a Chartered Engineer registered by the Engineering Council.



Xin Wu received the B.S. degree from the School of Software Engineering, South China University of Technology, Guangdong, China, where he is currently working toward the Ph.D. degree. His research interests include logical text generation and augmented language models.



Tao Wang received the Ph.D. degree in economics from the University of Southampton, Southampton, U.K., jointly trained with The Alan Turing Institute, London, U.K. He is currently a Research Fellow with the Department of Biostatistics and Health Informatics, King's College London, London, U.K. His research interests include text mining, social networks, and health informatics.